

SQL Assignment — Group 3: Playlist Table - Select Alias Filtering Aggregates

Group 3 — Playlist Table: Selects, Alias, Filtering, Aggregates

1. CREATE Playlist + INSERT 5 songs

Create a table called *Playlist* with columns: *id*, *song_name*, *artist*, *genre*, and *play_count*. Insert at least 5 sample records representing your favorite songs.

```
CREATE TABLE Playlist (  
  id          INT AUTO_INCREMENT PRIMARY KEY,  
  song_name   VARCHAR(100),  
  artist      VARCHAR(100),  
  genre       VARCHAR(50),  
  play_count  INT  
);  
  
INSERT INTO Playlist (song_name, artist, genre, play_count)  
VALUES  
  ('Blinding Lights', 'The Weeknd', 'Pop', 185),  
  ('Levitating',      'Dua Lipa',   'Pop', 142),  
  ('Sicko Mode',      'Travis Scott', 'Hip-Hop', 210),  
  ('God''s Plan',     'Drake',      'Hip-Hop', 95),  
  ('Perfect',         'Ed Sheeran', 'Romantic', 60);
```

Explanation: The data mixes multiple genres (Pop, Hip-Hop, Romantic) and a range of play counts, both above and below 100, so the filtering and aggregation queries in the next questions have varied results. Note the escaped apostrophe (God''s Plan) — in SQL, a literal single quote inside a string is written as two single quotes.

2. SELECT with column alias

Write a *SELECT* query to display only the *song_name* and *artist*, but show the *artist* column as 'Singer' using an alias.

```
SELECT song_name, artist AS Singer  
FROM Playlist;
```

Explanation: AS *Singer* renames the *artist* column only in the output/result set (it does not change the actual column name in the table). This is useful for making result sets more readable for a report or a front-end display.

3. SELECT with WHERE + ORDER BY

Find all songs where the *genre* is 'Pop' and *play_count* is greater than 100. Sort the results by *play_count* in descending order.

```
SELECT *  
FROM Playlist  
WHERE genre = 'Pop' AND play_count > 100  
ORDER BY play_count DESC;
```

Explanation: The *WHERE* clause filters to only Pop songs with more than 100 plays, and *ORDER BY play_count*

DESC arranges the matching rows from the most-played to the least-played.

4. COUNT aggregate function

Use the COUNT aggregate function to find out how many songs belong to the genre 'Hip-Hop'.

```
SELECT COUNT(*) AS hiphop_song_count
FROM Playlist
WHERE genre = 'Hip-Hop';
```

Explanation: COUNT(*) counts the number of rows returned after the WHERE genre = 'Hip-Hop' filter is applied, giving the total number of Hip-Hop songs in the table. The result column is aliased as hiphop_song_count for clarity.

5. SUM grouped by genre

Show the total number of plays (SUM of play_count) for each genre, grouped by genre.

```
SELECT genre, SUM(play_count) AS total_plays
FROM Playlist
GROUP BY genre;
```

Explanation: GROUP BY genre collects all rows sharing the same genre into a single group, and SUM(play_count) then adds up the play counts within each group, producing one total-plays row per distinct genre.